

Relatorio do trial

PARTICIPANTE

Luis Henrique Gonçalves Barbosa

KATA

Tarifador de Plano de Saúde

TRATAMENTO

Sem assistente de IA

TEMPO FINAL

20:40

TEMPO ESGOTADO

Nao

INICIO

16/09/2026, 17:10:00

FIM

16/09/2026, 17:30:40

RESULTADO E OBSERVACOES

8 de 8 testes unitários de aceitação passaram com sucesso (100%). Codificação manual.

ARQUIVOS ANEXADOS

4 arquivo(s) de código ou texto anexado(s) ao trial.

- texto-do-trial.txt (161 linhas)
- TarifadorPlanoSaude.java (110 linhas)
- CategoriaPlano.java (18 linhas)
- TipoAcomodacao.java (19 linhas)

texto-do-trial.txt

Participante: Luis Henrique Gonçalves Barbosa
Kata: Tarifador de Plano de Saúde
Tratamento: SEM_IA
Tempo: 20:40

Código implementado:

```
// === CategoriaPlano.java ===
package br.ufc.lab02.saude;

public enum CategoriaPlano {
    BASICO(1.00),
    ESPECIAL(1.30),
    PREMIUM(1.60);

    private final double fatorMultiplicador;

    CategoriaPlano(double fatorMultiplicador) {
        this.fatorMultiplicador = fatorMultiplicador;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }
}

// === TarifadorPlanoSaude.java ===
package br.ufc.lab02.saude;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class TarifadorPlanoSaude {

    private static final BigDecimal FAIXA_JOVEM = BigDecimal.valueOf(150.00);
    private static final BigDecimal FAIXA_ADULTO_1 = BigDecimal.valueOf(260.00);
    private static final BigDecimal FAIXA_ADULTO_2 = BigDecimal.valueOf(420.00);
    private static final BigDecimal FAIXA_IDOSO = BigDecimal.valueOf(720.00);
    private static final BigDecimal TAXA_ODONTOLOGIA = BigDecimal.valueOf(45.00);
    private static final BigDecimal FATOR_DESCONTO_COPARTICIPACAO =
BigDecimal.valueOf(0.85);

    public BigDecimal calcularMensalidade(
        int idade,
        CategoriaPlano categoria,
        TipoAcomodacao acomodacao,
        boolean incluiOdontologia,
        boolean possuiCoparticipacao) {

        validarDadosDoPlano(idade, categoria, acomodacao);

        BigDecimal mensalidadePorIdade = selecionarMensalidadePorIdade(idade);
        BigDecimal mensalidadeComCategoria = aplicarCategoria(mensalidadePorIdade,
categoria);
        BigDecimal mensalidadeComAcomodacao = adicionarAcomodacao(
            mensalidadeComCategoria,
            acomodacao
        );
    }
}
```

```

    );
    BigDecimal mensalidadeComOdontologia = adicionarOdontologia(
        mensalidadeComAcomodacao,
        incluiOdontologia
    );
    BigDecimal mensalidadeFinal = aplicarCoparticipacao(
        mensalidadeComOdontologia,
        possuiCoparticipacao
    );

    return formatarValor(mensalidadeFinal);
}

private void validarDadosDoPlano(
    int idade,
    CategoriaPlano categoria,
    TipoAcomodacao acomodacao) {

    if (idade < 0) {
        throw new IllegalArgumentException("Idade deve estar entre 0 e 120 anos.");
    }

    if (idade > 120) {
        throw new IllegalArgumentException("Idade deve estar entre 0 e 120 anos.");
    }

    if (categoria == null) {
        throw new IllegalArgumentException("Categoria do plano e acomodacao sao obrigatorias.");
    }

    if (acomodacao == null) {
        throw new IllegalArgumentException("Categoria do plano e acomodacao sao obrigatorias.");
    }
}

private BigDecimal selecionarMensalidadePorIdade(int idade) {
    if (idade <= 17) {
        return FAIXA_JOVEM;
    }

    if (idade <= 39) {
        return FAIXA_ADULTO_1;
    }

    if (idade <= 59) {
        return FAIXA_ADULTO_2;
    }

    return FAIXA_IDOSO;
}

private BigDecimal aplicarCategoria(BigDecimal valor, CategoriaPlano categoria) {
    BigDecimal fatorDaCategoria =
    BigDecimal.valueOf(categoria.getFatorMultiplicador());
    return valor.multiply(fatorDaCategoria);
}

private BigDecimal adicionarAcomodacao(BigDecimal valor, TipoAcomodacao acomodacao) {
    BigDecimal taxaDaAcomodacao = acomodacao.getTaxaAcomodacao();

```

```

        return valor.add(taxaDaAcomodacao);
    }

    private BigDecimal adicionarOdontologia(BigDecimal valor, boolean incluiOdontologia) {
        if (incluiOdontologia) {
            return valor.add(TAXA_ODONTOLOGIA);
        }

        return valor;
    }

    private BigDecimal aplicarCoparticipacao(BigDecimal valor, boolean possuiCoparticipacao) {
        if (possuiCoparticipacao) {
            return valor.multiply(FATOR_DESCONTO_COPARTICIPACAO);
        }

        return valor;
    }

    private BigDecimal formatarValor(BigDecimal valor) {
        return valor.setScale(2, RoundingMode.HALF_EVEN);
    }
}

```

```

// === TipoAcomodacao.java ===
package br.ufc.lab02.saude;

import java.math.BigDecimal;

public enum TipoAcomodacao {
    ENFERMARIA(BigDecimal.valueOf(0.00)),
    APARTAMENTO(BigDecimal.valueOf(90.00));

    private final BigDecimal taxaAcomodacao;

    TipoAcomodacao(BigDecimal taxaAcomodacao) {
        this.taxaAcomodacao = taxaAcomodacao;
    }

    public BigDecimal getTaxaAcomodacao() {
        return taxaAcomodacao;
    }
}

```

TarifadorPlanoSaude.java

```
package br.ufc.lab02.saude;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class TarifadorPlanoSaude {

    private static final BigDecimal FAIXA_JOVEM = BigDecimal.valueOf(150.00);
    private static final BigDecimal FAIXA_ADULTO_1 = BigDecimal.valueOf(260.00);
    private static final BigDecimal FAIXA_ADULTO_2 = BigDecimal.valueOf(420.00);
    private static final BigDecimal FAIXA_IDOSO = BigDecimal.valueOf(720.00);
    private static final BigDecimal TAXA_ODONTOLOGIA = BigDecimal.valueOf(45.00);
    private static final BigDecimal FATOR_DESCONTO_COPARTICIPACAO =
BigDecimal.valueOf(0.85);

    public BigDecimal calcularMensalidade(
        int idade,
        CategoriaPlano categoria,
        TipoAcomodacao acomodacao,
        boolean incluiOdontologia,
        boolean possuiCoparticipacao) {

        validarDadosDoPlano(idade, categoria, acomodacao);

        BigDecimal mensalidadePorIdade = selecionarMensalidadePorIdade(idade);
        BigDecimal mensalidadeComCategoria = aplicarCategoria(mensalidadePorIdade,
categoria);
        BigDecimal mensalidadeComAcomodacao = adicionarAcomodacao(
            mensalidadeComCategoria,
            acomodacao
        );
        BigDecimal mensalidadeComOdontologia = adicionarOdontologia(
            mensalidadeComAcomodacao,
            incluiOdontologia
        );
        BigDecimal mensalidadeFinal = aplicarCoparticipacao(
            mensalidadeComOdontologia,
            possuiCoparticipacao
        );

        return formatarValor(mensalidadeFinal);
    }

    private void validarDadosDoPlano(
        int idade,
        CategoriaPlano categoria,
        TipoAcomodacao acomodacao) {

        if (idade < 0) {
            throw new IllegalArgumentException("Idade deve estar entre 0 e 120 anos.");
        }

        if (idade > 120) {
            throw new IllegalArgumentException("Idade deve estar entre 0 e 120 anos.");
        }

        if (categoria == null) {
            throw new IllegalArgumentException("Categoria do plano e acomodacao sao
obrigatorias.");
        }
    }
}
```

```

    }

    if (acomodacao == null) {
        throw new IllegalArgumentException("Categoria do plano e acomodacao sao obrigatorias.");
    }
}

private BigDecimal selecionarMensalidadePorIdade(int idade) {
    if (idade <= 17) {
        return FAIXA_JOVEM;
    }

    if (idade <= 39) {
        return FAIXA_ADULTO_1;
    }

    if (idade <= 59) {
        return FAIXA_ADULTO_2;
    }

    return FAIXA_IDOSO;
}

private BigDecimal aplicarCategoria(BigDecimal valor, CategoriaPlano categoria) {
    BigDecimal fatorDaCategoria =
BigDecimal.valueOf(categoria.getFatorMultiplicador());
    return valor.multiply(fatorDaCategoria);
}

private BigDecimal adicionarAcomodacao(BigDecimal valor, TipoAcomodacao acomodacao) {
    BigDecimal taxaDaAcomodacao = acomodacao.getTaxaAcomodacao();
    return valor.add(taxaDaAcomodacao);
}

private BigDecimal adicionarOdontologia(BigDecimal valor, boolean incluiOdontologia) {
    if (incluiOdontologia) {
        return valor.add(TAXA_ODONTOLOGIA);
    }

    return valor;
}

private BigDecimal aplicarCoparticipacao(BigDecimal valor, boolean possuiCoparticipacao) {
    if (possuiCoparticipacao) {
        return valor.multiply(FATOR_DESCONTO_COPARTICIPACAO);
    }

    return valor;
}

private BigDecimal formatarValor(BigDecimal valor) {
    return valor.setScale(2, RoundingMode.HALF_EVEN);
}
}

```

CategoriaPlano.java

```
package br.ufc.lab02.saude;

public enum CategoriaPlano {
    BASICO(1.00),
    ESPECIAL(1.30),
    PREMIUM(1.60);

    private final double fatorMultiplicador;

    CategoriaPlano(double fatorMultiplicador) {
        this.fatorMultiplicador = fatorMultiplicador;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }
}
```

TipoAcomodacao.java

```
package br.ufc.lab02.saude;

import java.math.BigDecimal;

public enum TipoAcomodacao {
    ENFERMARIA(BigDecimal.valueOf(0.00)),
    APARTAMENTO(BigDecimal.valueOf(90.00));

    private final BigDecimal taxaAcomodacao;

    TipoAcomodacao(BigDecimal taxaAcomodacao) {
        this.taxaAcomodacao = taxaAcomodacao;
    }

    public BigDecimal getTaxaAcomodacao() {
        return taxaAcomodacao;
    }
}
```